SentinelSSH

Analyseur de logs SSH — Sécurité défensive

Stage 3 — Documentation technique

Portfolio Project — Technical Documentation (Stage 3)

Anil AKER
Holberton School Bordeaux
Projet de fin d'année — Demo Day / Titre RNCP niveau 5
github.com/anil-akr
1er octobre 2026

> **Le plan de construction du MVP.** Ce document décrit précisément ce que SentinelSSH doit faire, comment ses parties sont organisées et communiquent, comment le code sera géré et testé, et pourquoi chaque choix technique a été fait.

Sommaire

0. Des objectifs aux exigences techniques 3

1. User stories et maquettes 4

2. Architecture du système 8

3. Composants, classes et base de données 10

4. Diagrammes de séquence 15

5. Spécification des API 16

6. Gestion du code source et assurance qualité 18

7. Justifications techniques 20

Correspondance avec les consignes du Stage 3

Livrable demandé par Holberton	Où le trouver
Traduire les objectifs en plan technique	§0
1. User Stories and Mockups (Task 0)	§1
2. System Architecture (Task 1)	§2
3. Components, Classes, and Database Design (Task 2)	§3
4. Sequence Diagrams (Task 3)	§4
5. API Specifications (Task 4)	§5
6. SCM and QA Plans (Task 5)	§6
7. Technical Justifications (Task 6)	§7

0. Des objectifs aux exigences techniques

SentinelSSH est une application web qui analyse un fichier de journal de connexions SSH (`auth.log`), repère les adresses IP suspectes, leur attribue une gravité et présente les résultats à un administrateur connecté. Cette partie traduit les objectifs des Stages 1 et 2 en exigences précises, que les sections suivantes mettent en œuvre.

0.1 Exigences fonctionnelles (ce que l'application fait)

ID	Exigence	User stories
EF1	Se connecter avec un compte administrateur unique ; se déconnecter	US0
EF2	Envoyer un fichier .log ou .txt de 5 Mo maximum et obtenir son analyse	US1
EF3	Détecter les attaques avec 3 règles et attribuer une gravité : moyenne, élevée, critique	US2, US3
EF4	Afficher un résumé, un tableau trié par gravité et un graphique du top 10 des IP	US2, US6
EF5	Proposer, pour chaque IP élevée ou critique, une commande de blocage à copier (jamais exécutée)	US4
EF6	Conserver l'historique des analyses, permettre leur suppression, lister les IP récidivistes	US5, US7, US8
EF7	Informer sur les données traitées (page Confidentialité)	US10

0.2 Exigences non fonctionnelles (comment elle doit le faire)

Catégorie	Exigence	Comment on le vérifie
Sécurité	Connexion obligatoire, mot de passe haché, requêtes SQL paramétrées, affichage échappé	Tests 401, checklist XSS (§6.2)
Performance	Un fichier de 5 Mo est analysé en moins de 5 secondes ; 655 146 lignes réelles sans plantage	Mesure avec cli.py sur le dataset Loghub
Fiabilité	Au moins 80 % du code d'analyse couvert par des tests automatiques	Rapport pytest-cov dans la CI
Confidentialité	Minimisation des données (jamais le log complet), suppression possible, IP qui ne quittent pas la machine	Revue du schéma de base, route DELETE
Ergonomie	3 clics maximum de l'envoi au résultat ; lisible sur mobile ; gravité en couleur et en texte	Checklist manuelle (§6.2)
Maintenabilité	Une responsabilité par classe ; le cœur d'analyse ne dépend ni du web ni de la base	Diagramme de classes (§3.1)
Déploiement	Lancement en production en une commande, en suivant uniquement le README	Test sur un dossier vide (§6.3)

0.3 Contraintes

- Un seul développeur, 4 semaines de développement (sprints du 5 octobre au 1er novembre 2026).
- Une technologie JavaScript est exigée pour l'interface ; le cœur d'analyse reste en Python, le langage le mieux maîtrisé.
- Les compétences du titre RNCP 37674 « Développeur web et web mobile » doivent être démontrées.
- Usage local : l'application n'est pas mise en ligne sur Internet dans le MVP.

1. User stories et maquettes

1.1 User stories priorisées (MoSCoW)

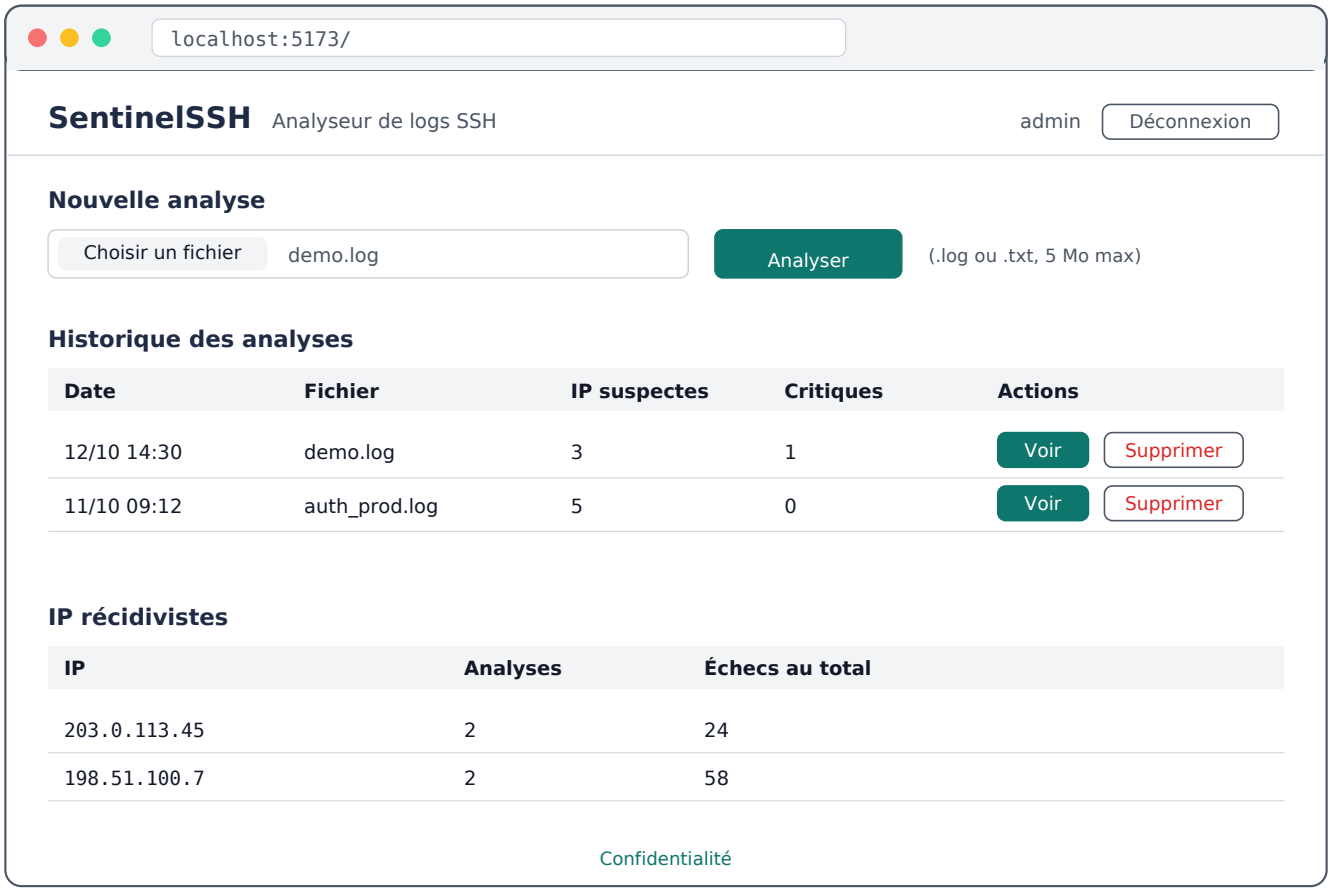
Format : « En tant que [type d'utilisateur], je veux [action], afin de [objectif] ». Deux types d'utilisateurs : l'**administrateur** (connecté) et le **visiteur** (non connecté). Priorités MoSCoW : **Must** (indispensable), **Should** (important), **Could** (bonus), **Won't** (pas dans cette version).

ID	User story	Critères d'acceptation	Priorité
US0	En tant qu'administrateur, je veux me connecter avec un compte protégé par mot de passe, afin que personne d'autre ne voie mes analyses.	Page de connexion · sans session, toute page protégée renvoie vers /login · bouton « Déconnexion »	**Must**
US1	En tant qu'administrateur, je veux envoyer mon fichier auth.log, afin d'obtenir une analyse sans lire les lignes à la main.	Fichier .log/.txt de 5 Mo max accepté · message clair sinon · ouverture du résultat	**Must**
US2	En tant qu'administrateur, je veux voir les IP suspectes classées par gravité, afin de traiter d'abord les plus dangereuses.	Tableau trié : critique, élevée, moyenne, puis nombre d'échecs	**Must**
US3	En tant qu'administrateur, je veux être alerté quand une IP réussit à se connecter après des échecs, afin de vérifier une compromission.	Gravité « critique » + règle R3 · compteur d'alertes critiques dans le résumé	**Must**
US4	En tant qu'administrateur, je veux copier la commande de blocage d'une IP, afin de la bloquer moi-même.	Commande iptables (ip6tables en IPv6) · bouton « Copier » et retour « Copié ! »	**Must**
US5	En tant qu'administrateur, je veux retrouver mes analyses passées, afin de suivre l'évolution des attaques.	Historique du plus récent au plus ancien · détail en un clic	**Must**
US6	En tant qu'administrateur, je veux voir un graphique des IP les plus actives, afin de comprendre la situation d'un coup d'œil.	Barres du top 10 des IP par nombre d'échecs	Should
US7	En tant qu'administrateur, je veux voir les IP qui reviennent dans plusieurs analyses, afin de repérer les attaquants persistants.	IP présentes dans au moins 2 analyses, avec le total d'échecs	Should
US8	En tant qu'administrateur, je veux supprimer une analyse, afin de ne pas garder de données inutiles.	Confirmation · l'analyse et ses IP disparaissent	Should
US9	En tant qu'administrateur, je veux utiliser l'application sur mon téléphone, afin de vérifier une alerte en déplacement.	Pages lisibles sur mobile, sans débordement horizontal	Should
US10	En tant que visiteur, je veux savoir quelles données sont conservées et combien de temps, afin de connaître mes droits.	Page « Confidentialité » accessible sans connexion	Should
US11	En tant qu'administrateur, je veux exporter une analyse en CSV, afin de l'archiver ou la partager.	Fichier CSV avec les colonnes du tableau	Could
US12	En tant qu'administrateur, je veux que l'outil bloque les IP tout seul, afin de ne rien avoir à faire.	Hors MVP : exigerait les droits root (voir §7)	Won't

1.2 Maquettes des écrans principaux

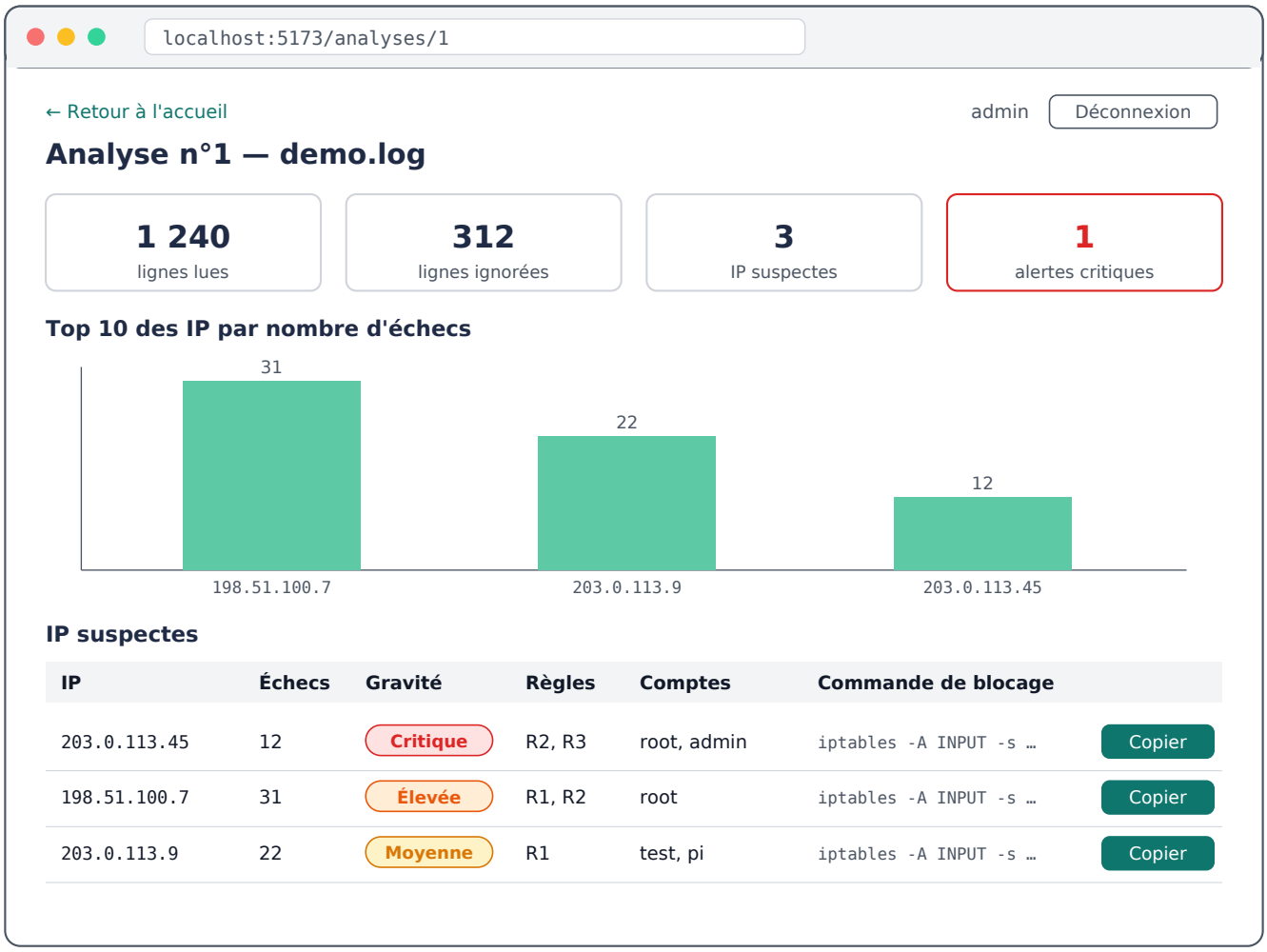
Ces maquettes (wireframes) montrent l'organisation des écrans et les éléments d'interface, pas les couleurs finales. La page « Confidentialité » est un simple texte et n'a pas de maquette.

Accueil — /



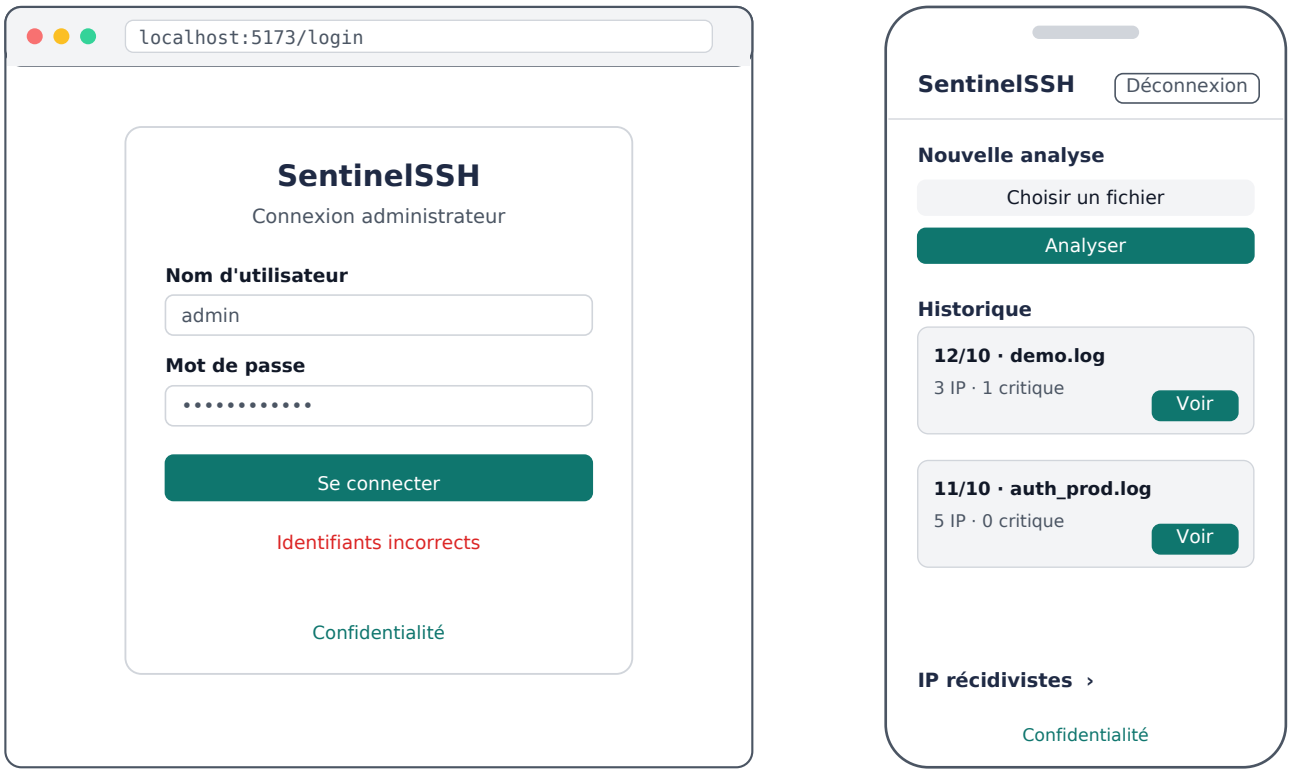
Envoi d'un fichier, historique des analyses et IP récidivistes.

Détail d'une analyse — /analyses/:id



Résumé en 4 chiffres, graphique, tableau avec gravité (couleur + texte) et commande à copier.

Connexion et affichage mobile



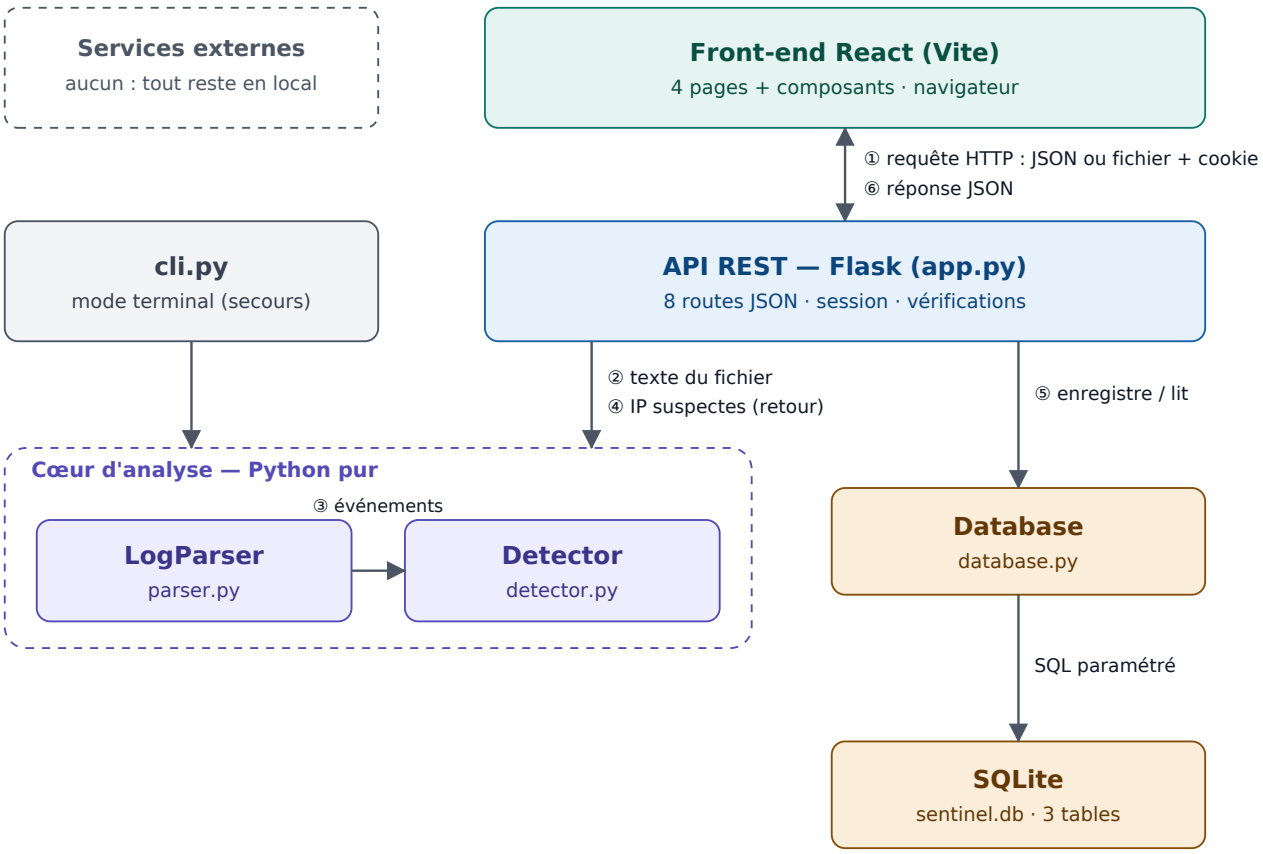
Connexion (ordinateur) Accueil (mobile)

Page de connexion avec son message d'erreur ; accueil sur téléphone, où les tableaux deviennent des cartes.

2. Architecture du système

2.1 Diagramme d'architecture et flux de données

L'application a 3 couches : le **front-end** React (ce que voit l'utilisateur), l'**API** Flask (la porte d'entrée du serveur) et le **back-end** (le cœur d'analyse et la base de données). Les numéros suivent le trajet d'une analyse.



Flèches pleines : qui appelle qui. Aucune donnée ne quitte la machine : il n'y a pas de service externe.

2.2 Trajet des données, étape par étape

- ① Le front envoie une requête HTTP à l'API : du JSON (connexion) ou un fichier (analyse). Le cookie de session l'accompagne automatiquement.
- ② L'API vérifie la session, la taille et le type du fichier, puis transmet son texte à `LogParser`.
- ③ `LogParser` transforme chaque ligne utile en objet `Event` ; `Detector` applique les 3 règles.
- ④ `Detector` renvoie à l'API la liste des `SuspectIP`, triée par gravité.
- ⑤ L'API demande à `Database` d'enregistrer l'analyse et ses IP dans SQLite, avec des requêtes paramétrées.
- ⑥ L'API répond en JSON ; React affiche le résumé, le graphique et le tableau.

2.3 Environnements

Environnement	Comment il tourne	À quoi il sert
Développement	Vite (port 5173) pour React + serveur Flask (port 5000) ; le proxy de Vite transmet `/api` à Flask	Coder et tester au quotidien
Pré-production	Front construit (`npm run build`), servi par Flask via Gunicorn, avec une base de test et `demo.log`	Vérifier la version avant de la déclarer stable
Production	Même montage, sur la machine de démo, avec le vrai compte administrateur et la `SECRET_KEY` de production	Le demo day

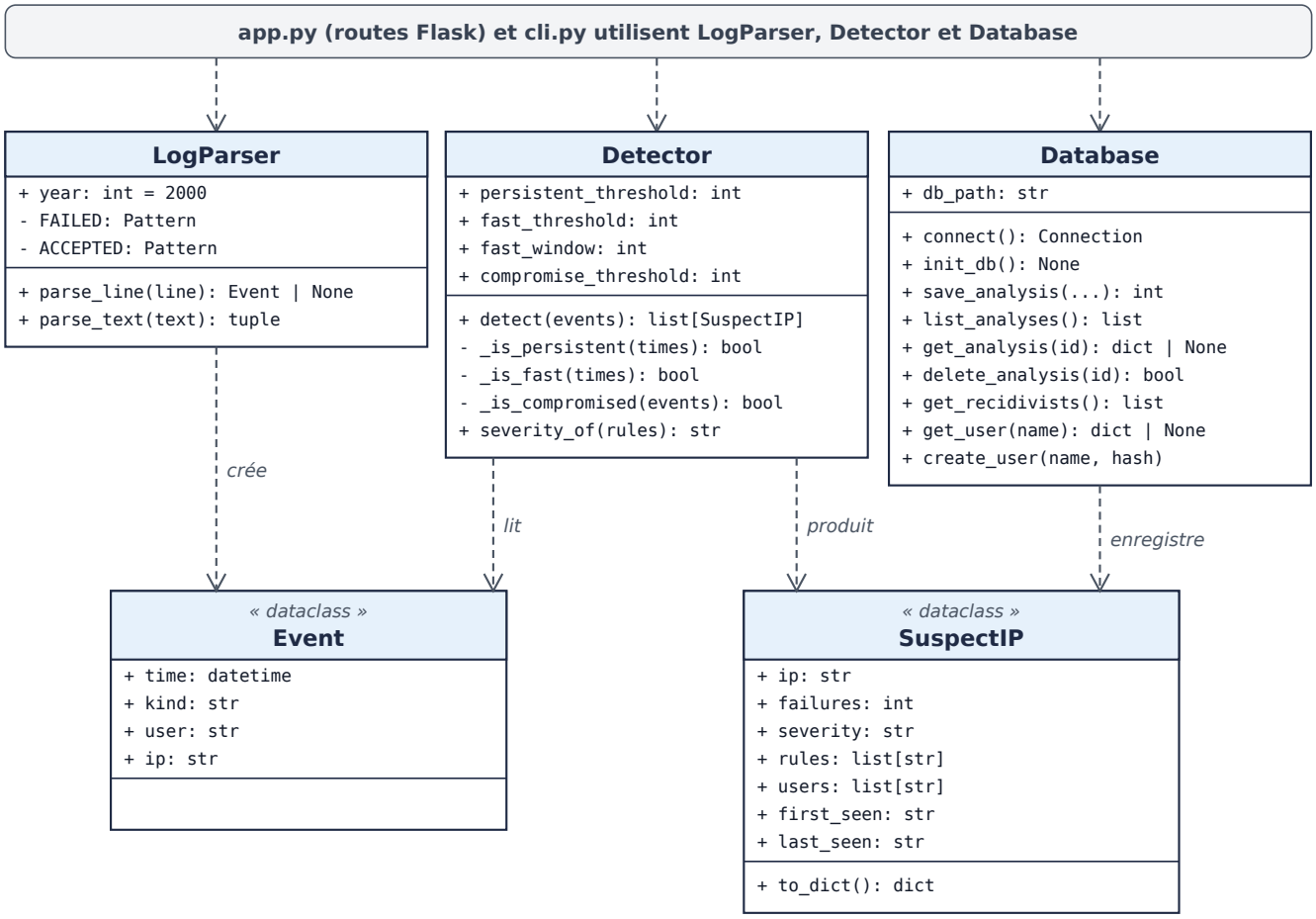
2.4 Organisation des fichiers

```
sentinelssh/
├── backend/
│   ├── app.py              # routes de l'API, connexion, erreurs, commande create-admin
│   ├── cli.py              # mode terminal de secours
│   ├── schema.sql          # création des 3 tables
│   ├── requirements.txt    # flask, python-dotenv, gunicorn, pytest, pytest-cov, pycodestyle
│   ├── .env.example        # modèle de configuration (le vrai .env n'est jamais commité)
│   ├── sentinel/
│   │   ├── config.py       # seuils, taille max, chemin de la base, SECRET_KEY
│   │   ├── models.py       # Event, SuspectIP
│   │   ├── parser.py       # LogParser
│   │   ├── detector.py     # Detector
│   │   └── database.py     # Database
│   └── tests/              # test_parser.py, test_detector.py, test_database.py, test_api.py
├── frontend/
│   ├── vite.config.js      # proxy /api -> Flask (développement)
│   └── src/                # App.jsx, api.js, pages/, components/, styles.css
├── demo/demo.log           # fichier de démonstration (adresses fictives)
├── docs/                   # rapports, journal des décisions
└── .github/                # workflow de CI, modèle de Pull Request
```

3. Composants, classes et base de données

3.1 Back-end : diagramme de classes

Le back-end est découpé en 5 classes, chacune avec une seule responsabilité. Deux sont des **dataclasses** : des classes qui servent surtout à ranger des données, dont Python écrit lui-même le constructeur.



Notation UML : + public, - privé (convention Python : nom commençant par _). Flèche pointillée : « utilise ».

3.2 Description des classes et modules

Élément	Attributs principaux	Méthodes et rôle
`Event` (dataclass)	time, kind (« fail » ou « success »), user, ip	Une tentative de connexion lue dans le journal. Pas de méthode : c'est un conteneur de données.
`SuspectIP` (dataclass)	ip, failures, severity, rules, users, first_seen, last_seen	`to_dict()` : convertit l'objet en dictionnaire, pour le JSON et la base.
`LogParser`	year (année bissextile fixe, 2000) ; expressions régulières des lignes « Failed » et « Accepted »	`parse_line(line)` : renvoie un `Event` ou `None`. `parse_text(text)` : renvoie la liste des événements, le nombre de lignes lues et ignorées.
`Detector`	Les 4 seuils des règles, lus dans `config.py`	`detect(events)` : regroupe par IP, applique les règles, renvoie les `SuspectIP` triées. `_is_persistent`, `_is_fast`, `_is_compromised` : une méthode privée par règle (R1, R2, R3). `severity_of(rules)` : gravité la plus haute parmi les règles déclenchées.
`Database`	db_path (chemin du fichier SQLite, remplaçable dans les tests)	`connect()` ouvre la base et active les clés étrangères · `init_db()` crée les tables · `save_analysis`, `list_analyses`, `get_analysis`, `delete_analysis`, `get_recidivists` · `get_user`, `create_user` (compte administrateur)
`app.py` (module)	L'application Flask, sa configuration	Les 8 routes de l'API · décorateur `login_required` (401 sans session) · `block_command(ip)` · gestionnaires d'erreurs 400, 401, 404, 413 · commande `create-admin` · service du front construit en production
`cli.py` (module)	—	`main()` : analyse un fichier passé en argument et affiche un tableau texte, sans base de données.

> **Pourquoi ce découpage :** `LogParser` et `Detector` ne connaissent ni Flask ni SQLite. On peut donc les tester seuls, et `cli.py` les réutilise tels quels. `Detector` reçoit ses seuils à la création : un test peut en choisir d'autres sans modifier le code.

3.3 Front-end : composants et interactions

```
App                          (déclare les routes)
├── LoginPage                "/login"            (publique)
├── PrivacyPage              "/confidentialite"  (publique)
└── ProtectedRoute           (vérifie la session, sinon renvoie vers /login)
    ├── Header               (nom de l'admin + bouton Déconnexion)
    ├── HomePage             "/"
    │   ├── UploadForm
    │   ├── HistoryTable
    │   └── RecidivistsTable
    └── AnalysisPage         "/analyses/:id"
        ├── SummaryCards
        ├── TopIpsChart
        └── SuspectsTable
            ├── SeverityBadge
            └── CopyButton
```

Composant	Rôle et interactions	Reçoit (props)	Garde (state)
`LoginPage`	Envoie nom + mot de passe via `login()`, puis ouvre l'accueil ; affiche « Identifiants incorrects »	—	username, password, error

Composant	Rôle et interactions	Reçoit (props)	Garde (state)
`ProtectedRoute`	Appelle `me()` : affiche les pages protégées si la session est valide, sinon redirige	—	status
`Header`	Nom de l'admin, bouton Déconnexion (`logout()`)	username	—
`HomePage`	Charge historique et récidivistes ; les recharge après une suppression	—	analyses, recidivists, error
`UploadForm`	Envoie le fichier (`uploadLog()`), puis ouvre la page de détail	—	file, loading, error
`HistoryTable`	Liste des analyses ; Voir / Supprimer (avec confirmation)	analyses, onDelete	—
`RecidivistsTable`	Liste des IP récidivistes	recidivists	—
`AnalysisPage`	Lit l'id dans l'URL et charge l'analyse (`getAnalysis()`)	—	analysis, loading, error
`SummaryCards`, `TopIpsChart`	Les 4 chiffres du résumé ; graphique du top 10 (Recharts)	analysis / suspects	—
`SuspectsTable`	Tableau des IP ; utilise `SeverityBadge` et `CopyButton`	suspects	—
`CopyButton`	Copie la commande (presse-papiers) et affiche « Copié ! »	text	copied
`PrivacyPage`	Texte fixe : données traitées, finalité, durée de conservation, droits	—	—

Toutes les requêtes passent par un seul fichier, `api.js` : `login`, `logout`, `me`, `uploadLog`, `listAnalyses`, `getAnalysis`, `deleteAnalysis`, `listRecidivists`. Sur une réponse 401, l'utilisateur est renvoyé vers `/login`. **Props** : données reçues du parent ; **state** : données gardées par le composant, qui le réaffichent quand elles changent.

3.4 Base de données : diagramme entité-relation

Base relationnelle SQLite, 3 tables. `analyses` et `suspect_ips` sont reliées en « 1 → N » : une analyse contient zéro, une ou plusieurs IP suspectes. `users` contient le compte administrateur.

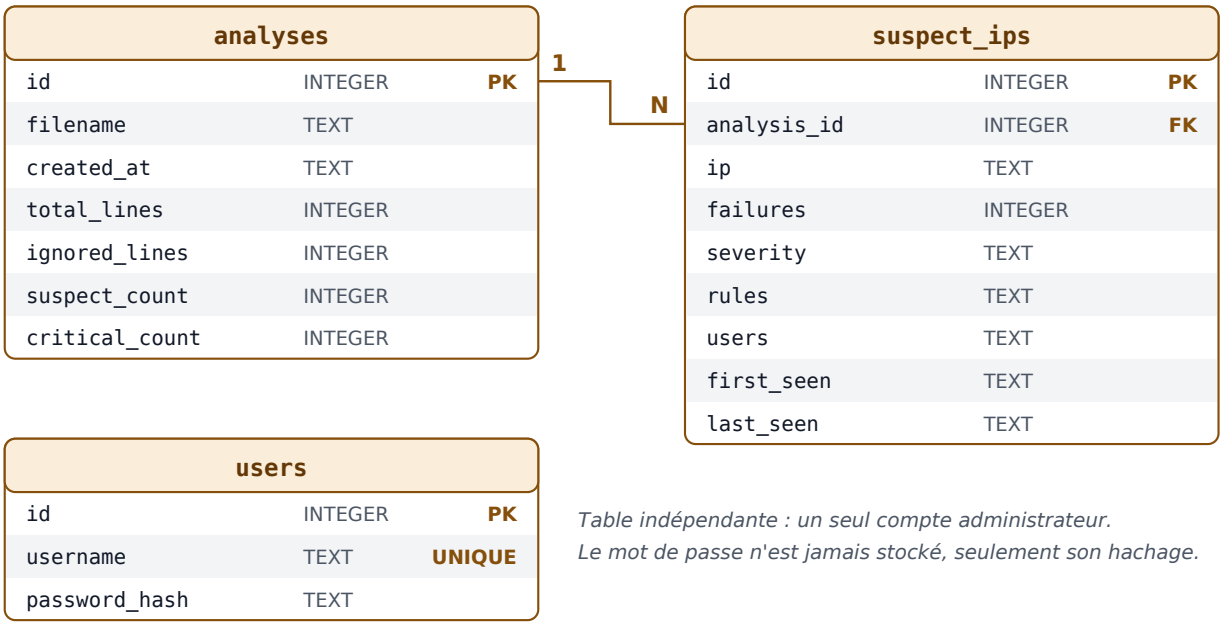


Table indépendante : un seul compte administrateur.
Le mot de passe n'est jamais stocké, seulement son hachage.

PK : clé primaire. FK : clé étrangère vers analyses.id, avec suppression en cascade.

3.5 Schéma détaillé

Table analyses — une ligne par fichier analysé

Colonne	Type	Contraintes	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identifiant de l'analyse
filename	TEXT	NOT NULL	Nom du fichier envoyé
created_at	TEXT	NOT NULL	Date de l'analyse, format ISO 8601
total_lines / ignored_lines	INTEGER	NOT NULL	Lignes lues / lignes non reconnues
suspect_count / critical_count	INTEGER	NOT NULL	Nombre d'IP suspectes / d'alertes critiques

Table suspect_ips — une ligne par IP suspecte dans une analyse

Colonne	Type	Contraintes	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identifiant
analysis_id	INTEGER	NOT NULL, FOREIGN KEY → analyses(id) ON DELETE CASCADE	Analyse à laquelle appartient l'IP
ip	TEXT	NOT NULL	Adresse IPv4 ou IPv6
failures	INTEGER	NOT NULL	Nombre d'échecs de connexion
severity	TEXT	NOT NULL, CHECK (moyenne, elevee, critique)	Gravité
rules / users	TEXT	—	Règles déclenchées (ex. « R2,R3 ») / 3 comptes les plus visés
first_seen / last_seen	TEXT	—	Première et dernière tentative (heure du journal)

Table users — le compte administrateur

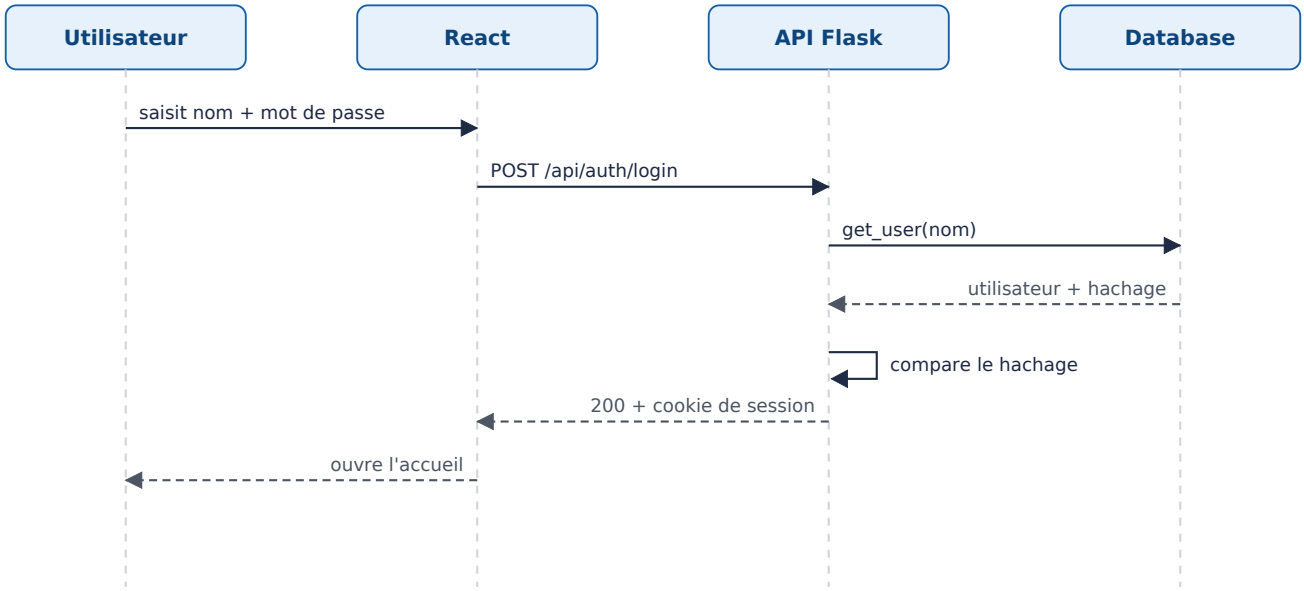
Colonne	Type	Contraintes	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identifiant
username	TEXT	NOT NULL, UNIQUE	Nom de connexion
password_hash	TEXT	NOT NULL	Hachage du mot de passe (Werkzeug), jamais le mot de passe lui-même

- **Clés étrangères :** SQLite les désactive par défaut. `Database.connect()` exécute `PRAGMA foreign_keys = ON` à chaque connexion, sinon la cascade ne fonctionne pas.
- **Minimisation :** seules les IP suspectes et des compteurs sont stockés, jamais le contenu du journal.
- **Récidivistes :** une requête regroupe `suspect_ips` par IP (`GROUP BY`) et garde celles présentes dans au moins 2 analyses (`HAVING`).

4. Diagrammes de séquence

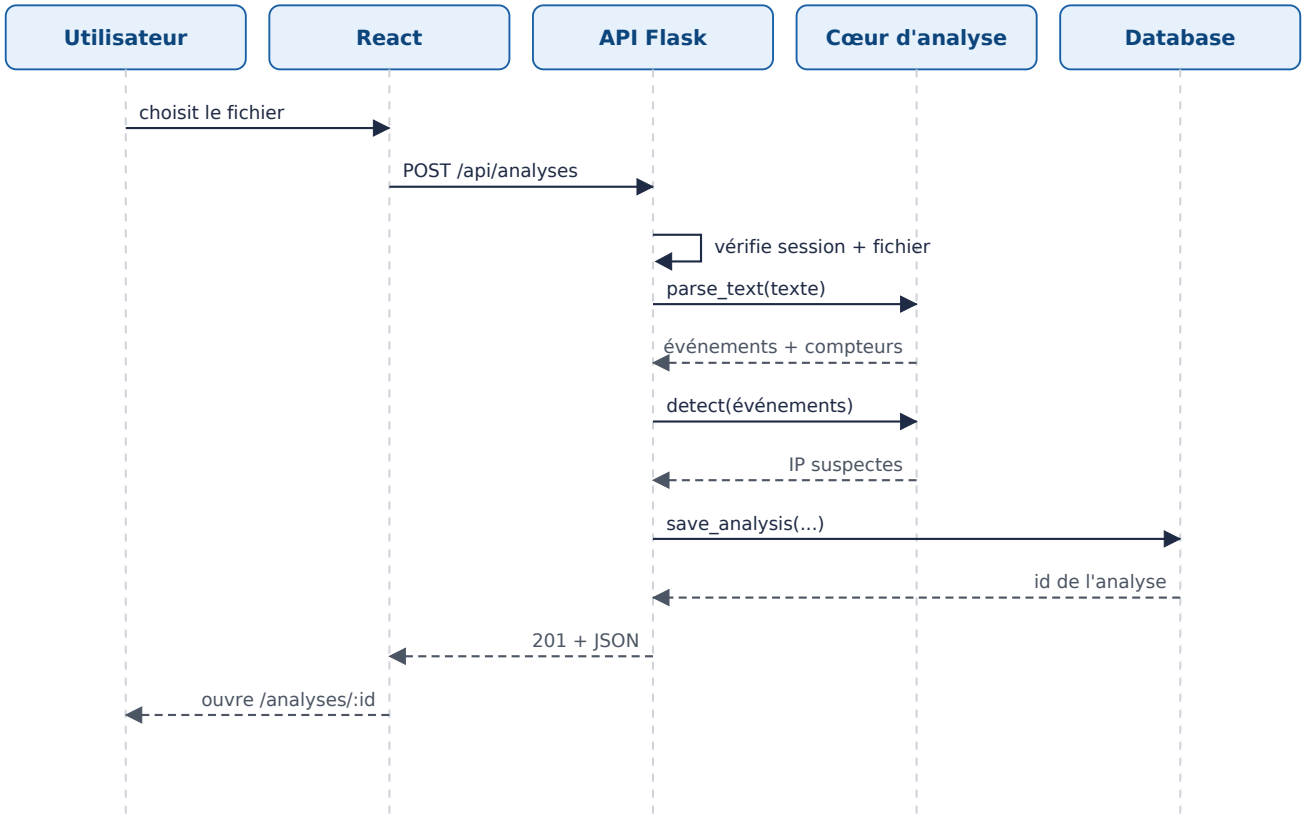
Trois cas d'usage clés. Flèche pleine : appel ; flèche en pointillés : réponse.

4.1 L'administrateur se connecte



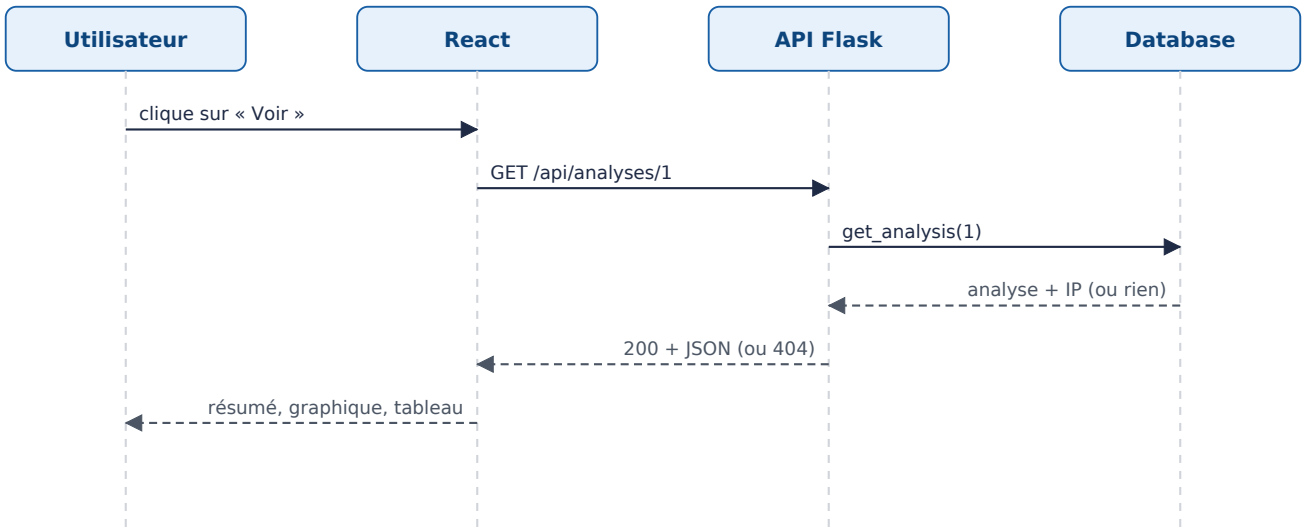
Si le nom ou le mot de passe est faux : 401 avec le même message dans les deux cas.

4.2 L'administrateur envoie un fichier à analyser



Cœur d'analyse = LogParser puis Detector. Fichier refusé : 400 ou 413 ; session absente : 401.

4.3 L'administrateur consulte une analyse



La suppression suit le même chemin avec DELETE : l'API répond 204, puis React recharge l'historique.

5. Spécification des API

5.1 API externes

Le MVP n'utilise aucune API web externe. C'est un choix volontaire :

- **Confidentialité :** les adresses IP sont des données personnelles ; elles ne quittent jamais la machine.
- **Fiabilité :** l'application fonctionne sans connexion Internet et ne dépend d'aucun service tiers le jour de la démo.
- **Sécurité :** aucune clé d'API à protéger, aucune limite de requêtes à gérer.

Ressource externe	Utilisation	Statut
API du navigateur : Fetch et Clipboard	Appeler l'API interne ; copier la commande de blocage	Utilisées (intégrées au navigateur, sans compte)
Dataset OpenSSH de Loghub	Fichier de test téléchargé une fois (655 146 lignes réelles) ; l'application ne l'appelle pas	Utilisé pour les tests de robustesse
AbuseIPDB (réputation des IP)	Savoir si une IP est déjà signalée ailleurs	Écartée du MVP : enverrait les IP à un tiers ; piste pour une version future

5.2 API interne : conventions

- Toutes les routes commencent par `/api` et échangent du **JSON** (sauf l'envoi de fichier, en `multipart/form-data`).
- **Authentification** par cookie de session (HttpOnly, SameSite=Lax), créé par `POST /api/auth/login`. Sans session, les routes protégées répondent **401**.
- **Erreurs** toujours au même format : `{"error": "message clair"}`.
- **Codes HTTP :** 200 lecture réussie · 201 création · 204 suppression · 400 entrée invalide · 401 non connecté · 404 introuvable · 413 fichier trop gros.

5.3 Liste des endpoints

Méthode	URL	Session	Entrée	Sortie en cas de succès
POST	`/api/auth/login`	Non	JSON `{username, password}`	200 `{username}` + cookie
POST	`/api/auth/logout`	Non	—	204
GET	`/api/auth/me`	Oui	—	200 `{username}`
POST	`/api/analyses`	Oui	Fichier (champ `file`)	201 `{id, filename, suspect_count, critical_count}`
GET	`/api/analyses`	Oui	—	200 liste des analyses (sans les IP)
GET	`/api/analyses/<id>`	Oui	id dans l'URL	200 analyse + liste `suspects`
DELETE	`/api/analyses/<id>`	Oui	id dans l'URL	204
GET	`/api/recidivists`	Oui	—	200 liste `{ip, analyses_count, total_failures}`

5.4 Détail des entrées et sorties

Connexion

```
POST /api/auth/login
Entrée  : {"username": "admin", "password": "••••••••••••"}
200     : {"username": "admin"}               + Set-Cookie: session=... ; HttpOnly ; SameSite=Lax
400     : {"error": "Nom d'utilisateur et mot de passe obligatoires"}
401     : {"error": "Identifiants incorrects"}

POST /api/auth/logout      -> 204 (session supprimée)
GET  /api/auth/me          -> 200 {"username": "admin"}   |   401 {"error": "Connexion requise"}
```

Analyses

```
POST /api/analyses          (multipart/form-data, champ "file")
201 : {"id": 1, "filename": "demo.log", "suspect_count": 3, "critical_count": 1}
400 : {"error": "Extension non autorisée : .log ou .txt uniquement"}
413 : {"error": "Fichier trop volumineux (5 Mo maximum)"}

GET /api/analyses
200 : [{"id": 1, "filename": "demo.log", "created_at": "2026-10-12T14:30:00",
        "total_lines": 1240, "suspect_count": 3, "critical_count": 1}]

DELETE /api/analyses/1      -> 204   |   404 {"error": "Analyse introuvable"}

GET /api/recidivists
200 : [{"ip": "198.51.100.7", "analyses_count": 2, "total_failures": 58}]
```

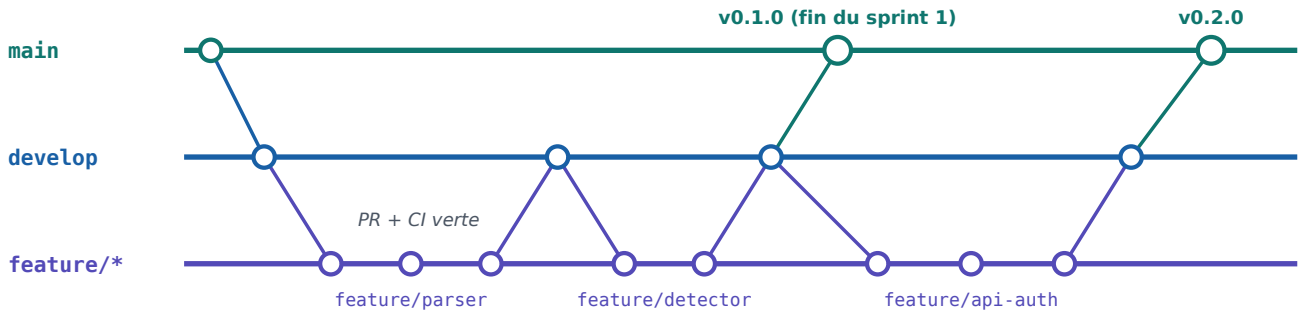
Détail d'une analyse

```
GET /api/analyses/1
200 : {
  "id": 1, "filename": "demo.log", "created_at": "2026-10-12T14:30:00",
  "total_lines": 1240, "ignored_lines": 312, "suspect_count": 3, "critical_count": 1,
  "suspects": [
    {"ip": "203.0.113.45", "failures": 12, "severity": "critique",
     "rules": ["R2", "R3"], "users": ["root", "admin", "anil"],
     "first_seen": "Oct 12 14:03:21", "last_seen": "Oct 12 14:05:02",
     "block_command": "iptables -A INPUT -s 203.0.113.45 -j DROP"}
  ]
}
404 : {"error": "Analyse introuvable"}
```

6. Gestion du code source et assurance qualité

6.1 Stratégie de gestion du code (SCM)

Outils : **Git** pour les versions, **GitHub** pour héberger le dépôt, les Pull Requests, les issues et le tableau de tâches (GitHub Projects). La stratégie de branches suit un modèle simple à trois niveaux :



Chaque cercle est un commit. Les fusions vers develop passent par une Pull Request ; develop rejoint main à chaque fin de sprint.

Branche	Rôle	Règle
`main`	Version stable, toujours présentable	Protégée : elle ne reçoit que `develop`, en fin de sprint, avec un tag de version
`develop`	Intégration des fonctionnalités terminées	Ne reçoit que des Pull Requests dont la CI est verte
`feature/…`, `fix/…`	Une branche par tâche ou user story (ex. `feature/parser`)	Créée depuis `develop`, supprimée après la fusion

- **Commits :** petits et fréquents (au moins un par session de travail), au format *Conventional Commits* : `feat:`, `fix:`, `test:`, `docs:`, `refactor:`. Exemple : `feat: add R2 fast brute-force rule`.
- **Pull Requests et revue de code :** chaque branche revient dans `develop` par une PR. Un modèle de PR impose une checklist : tests écrits et verts, style respecté, aucun secret dans le code, documentation à jour. Le projet étant solo, la relecture est faite ligne par ligne avant fusion ; les PR sensibles (connexion, base de données) sont montrées à un camarade quand c'est possible.
- **Versions :** un tag par fin de sprint (`v0.1.0` à `v0.3.0`), puis `v1.0.0` au gel du 1er novembre.
- **Suivi :** une issue par user story ou tâche, liée à sa PR ; kanban À faire / En cours / Terminé.
- **Fichiers exclus** (`.gitignore`) : `venv/`, `node_modules/`, `dist/`, `*.db`, `.env`, vrais journaux.

6.2 Stratégie de tests et de qualité (QA)

Type de test	Outil	Ce qui est testé	Quand
Tests unitaires	pytest	`LogParser` (chaque format de ligne, IPv6, ligne inutile) · `Detector` (chaque règle, seuils, gravité) · `Database` (cascade, récidivistes)	À chaque commit ; CI
Tests d'intégration	pytest + client de test Flask, base temporaire	Les 8 routes : codes HTTP, JSON, 401 sans session, 400/404/413	CI sur chaque PR
Tests manuels de l'API	Postman (ou curl)	Une collection des 8 routes, rejouée avant de brancher le front	Fin du sprint 2
Tests de bout en bout	Checklist manuelle dans le navigateur	Les 13 scénarios ci-dessous, sur ordinateur et en mode mobile	Avant chaque fusion dans main
Qualité du code	pycodestyle (Python), ESLint (JavaScript)	Respect du style, erreurs courantes	CI sur chaque PR
Couverture	pytest-cov	Au moins 80 % du package `sentinel`	CI
Robustesse	`cli.py` + dataset Loghub	655 146 lignes sans plantage ; 5 Mo en moins de 5 s	Sprint 4

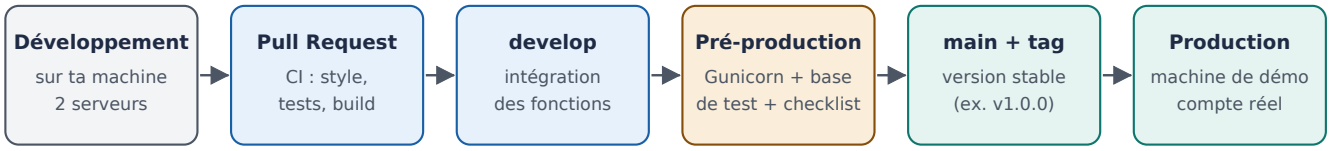
Checklist de bout en bout

#	Scénario	Résultat attendu
1	Ouvrir l'accueil sans être connecté	Redirection vers la page de connexion
2	Se connecter avec un mauvais mot de passe	« Identifiants incorrects »
3	Envoyer `demo.log`	Les 3 niveaux de gravité apparaissent
4	Envoyer un fichier `.exe` / de plus de 5 Mo / vide	Message d'erreur clair / message d'erreur clair / « Aucune IP suspecte »
5	Log contenant l'utilisateur `<script>alert(1)</script>`	Affiché comme du texte, aucune alerte
6	Cliquer sur « Copier »	« Copié ! », commande dans le presse-papiers
7	Supprimer une analyse	Elle disparaît après confirmation
8	Envoyer 2 fois le même fichier	Les IP apparaissent dans les récidivistes
9	Recharger la page de détail	La page s'affiche toujours
10	Arrêter le serveur puis recharger	Message d'erreur clair, pas de page blanche
11	Chaque page en mode mobile	Lisible, sans débordement
12	Se déconnecter puis revenir en arrière	Retour à la page de connexion
13	Ouvrir /confidentialite sans être connecté	La page s'affiche

- **Intégration continue (GitHub Actions)**, à chaque push et Pull Request vers `develop` ou `main` : installation des dépendances, pycodestyle, pytest avec couverture, puis ESLint et construction du front. Une étape en échec bloque la fusion.
- **Définition de « terminé » :** code fusionné par PR, tests écrits et verts, checklist concernée validée, issue fermée, README à jour.

6.3 Pipeline de déploiement

Le code passe par les mêmes étapes jusqu'à la production. La pré-production (staging) reproduit la production sur la machine de développement, avec une base de test.



Un blocage à une étape (CI rouge, checklist non validée) arrête la progression vers la suivante.

- **Pré-production :** `npm run build`, puis `gunicorn -b 127.0.0.1:8000 app:app` avec une base de test et `demo.log`. La checklist complète doit passer avant de fusionner dans `main`.
- **Production :** sur la machine de démo, à partir du tag `v1.0.0` : installation en suivant uniquement le README, `.env` avec la `SECRET_KEY` de production, création du compte administrateur, lancement avec Gunicorn.
- **Retour arrière :** si une version pose problème, on relance le tag précédent (`git checkout v0.3.0`).

7. Justifications techniques

Chaque choix répond à une exigence du §0, à une contrainte ou à une recommandation reconnue.

Décision	Alternatives écartées	Exigence ou contrainte	Justification
Python + Flask (back)	Node.js/Express, Django, FastAPI	Délai de 4 semaines ; maintenabilité	Python est le langage le mieux maîtrisé, et la détection (le cœur) y est écrite. Flask est le cadre le plus léger ; Django serait trop lourd, FastAPI ajouterait des notions.
React + Vite (front)	HTML/CSS seul, Vue, Angular	Techno JavaScript exigée ; compétence RNCP « partie dynamique »	React est le standard le plus répandu. Vite crée et lance un projet en une commande.
SQLite + SQL écrit à la main	PostgreSQL, MongoDB, ORM SQLAlchemy	Usage local ; données reliées (1 → N) ; sécurité	Pas de serveur de base à installer. Écrire le SQL permet de comprendre chaque requête et de montrer les requêtes paramétrées contre l'injection.
Classes à responsabilité unique	Un seul gros fichier	Maintenabilité ; testabilité	Le cœur d'analyse ne dépend ni du web ni de la base : testable seul et réutilisé par le mode terminal.
Session Flask + hachage Werkzeug	Jeton JWT, authentification « maison »	Sécurité ; simplicité	Outils éprouvés, fournis avec Flask. Le cookie HttpOnly est inaccessible au JavaScript, contrairement à un jeton stocké par le front.
3 niveaux de gravité	5 niveaux	Lisibilité pour l'administrateur	Un niveau par règle, une action par niveau : surveiller, bloquer, vérifier une compromission. Un niveau sans règle ni action ne servirait à rien.
Blocage suggéré, pas exécuté	Blocage automatique	Sécurité : pas de droits root	L'application n'a pas besoin de privilèges ; la décision reste humaine, ce qui la distingue de fail2ban.
Analyse à la demande	Surveillance en temps réel	Délai de 4 semaines	Périmètre réaliste pour le MVP ; le temps réel est prévu pour une version future.

Décision	Alternatives écartées	Exigence ou contrainte	Justification
Aucune API externe	AbuseIPDB	Confidentialité (RGPD) ; fiabilité de la démo	Les IP ne quittent pas la machine ; aucune dépendance à un service tiers.
Gunicorn + front construit	Serveurs de développement, Docker	Déploiement en une commande ; compétence RNCP « déploiement »	Plus robuste pour la démo. Docker ajouterait un outil à apprendre pendant le projet.
main / develop / feature	Tout sur main	Qualité ; version présentable à tout moment	main reste stable pendant que develop avance ; chaque fusion passe par une PR et la CI.
pytest, pycodestyle, ESLint, GitHub Actions	Tests uniquement manuels	Fiabilité (80 % de couverture)	Les tests se lancent seuls à chaque envoi de code : une régression est vue immédiatement.

Recommandations suivies

- **OWASP Top 10** (référence mondiale des risques des applications web) : protections contre les catégories « injection » et « défaillances d'identification et d'authentification ».
- **RGPD :** une adresse IP est une donnée personnelle ; base légale d'intérêt légitime pour la sécurité des réseaux (considérant 49), minimisation, droit à l'effacement, information via la page Confidentialité.
- **Référentiel du titre RNCP 37674 :** maquettes, interface dynamique, base relationnelle, composants métier, application sécurisée, déploiement documenté.